

Engineering Review Document

What worked well in your architecture?

Decoupling the modules using intermediate queues (even simple file-based ones) proved highly effective. It allowed each component (Ingestion, TI, ML, SOAR) to be developed, tested, and run independently. The Streamlit dashboard was also very easy to integrate since it only needed to read the final alerts queue.

What would you do differently?

If doing this again for a production environment, I would immediately start with Docker Compose and RabbitMQ/Redis. While file-based queues are great for a standalone demo MVP, handling file locks and read pointers manually in Python can lead to race conditions under heavy load.

Performance: How fast does it process events?

In its current state, the platform processes events near-instantaneously (sub-millisecond per log) because the TI API is mocked and the ML model is small. Streamlit updates every 2 seconds by design to prevent UI flicker.

Scalability: How would you handle 10x more data?

1. Message Broker: Switch to Apache Kafka for high-throughput log queuing.
2. Horizontal Scaling: Deploy multiple instances of the ML detector and TI enricher using Kubernetes to process messages concurrently from Kafka consumer groups.
3. Database: Use Elasticsearch for storing alerts so the dashboard can query aggregated metrics.

Security: What are the vulnerabilities in your platform?

1. API Key Management: In production, API keys would need to be securely managed.
2. File Permissions: The JSONL queues are stored on disk. If the host machine is compromised, attackers could tamper with the logs.
3. Unauthenticated Dashboard: The Streamlit dashboard lacks authentication, exposing security alerts.